Predicting Keystrokes from Electromyography Signals

Maitreyi Pareek
maitreyi@g.ucla.edu

Farrah Galal
fgalal@g.ucla.edu

Aki Hasegawa-Johnson
aki0501@g.ucla.edu

Abstract

We studied how to lower character error rate (CER) when training and testing on a single user from the *emg2qwerty* wrist-sEMG typing benchmark. Starting from the provided CNN+CTC baseline, we tested several architecture changes and training choices, including CNN+BiGRU, CNN+LSTM, and CNN+Transformer Encoder, along with data augmentation, learning-rate and batch-size tuning, and ablations on channel count, training data amount, and sampling rate. Overall, recurrent models performed best: CNN+BiGRU achieved the lowest CER (17.12 in our best setting), while the Transformer variant did not generalize as well to the test session. We also found that results were sensitive to optimization and input information: augmentation improved robustness in some settings, the best learning rate in our sweep was 10^{-3} with batch size 16, and reducing channels, training data, or sampling rate consistently increased CER. Overall, our experiments suggest that the best performance in the personalized setting comes from pairing a model that captures local EMG patterns with one that models longer-range temporal context, while keeping enough spatial (channel) and temporal (sampling) detail in the input signal.

1 Introduction

Because this project is based on emg2qwerty, the central question is not whether wrist sEMG can support typing in principle, but rather which modeling choices most effectively reduce CER in the benchmark setting we studied. The original emg2qwerty paper already established a strong ASR-style baseline and showed that the task is both structured and challenging. It introduced the largest public wrist-sEMG typing dataset to date, framed the task as sequence transduction from continuous multichannel signals to characters, and reported that the generic model's performance on unseen users remained above 50% CER. That result makes personalization and model adaptation especially important, since a system that performs poorly on a new user but improves substantially with user-specific training may still be practically valuable.

Our project therefore focused on, in a personalized single-subject setting, whether we can improve the baseline by replacing the original TDS-based architecture with alternative sequence models and by tuning the training pipeline more aggressively. This question is well motivated by the design of the original baseline itself. In this paper [1], Sivakumar argues that large receptive fields matter for emg2qwerty because nearby characters affect keystroke-related sEMG through co-articulation. They also explicitly note that architectures such as RNNs and Transformers have been effective in ASR, but were not explored in that baseline study. That left a natural opening for our project: if co-articulation and longer temporal dependencies are important, then a model with an explicit

[0]Code repository: `https://github.com/maitreyi03/emg2qwerty`

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

recurrent or attention-based temporal component might outperform a purely convolutional baseline in the personalized regime.

2 Methods

Our project investigated how architectural choice, data augmentation, and training configuration affect EMG-to-text recognition performance. We compared several sequence modeling approaches built on top of the EMG input representation, beginning with a baseline system and then evaluating alternative architectures including BiGRU, LSTM, and Transformer Encoder variants. In addition, we studied whether performance could be improved through data augmentation, and we ran both hyperparameter tuning and ablation experiments to better understand which factors were most important for achieving low character error rate (CER). This experimental design was motivated by the idea that good performance in EMG-based recognition likely depends on both the model's ability to capture temporal structure and the amount of useful signal information preserved in the input.

2.1 Baseline Model

We began with a baseline model that served as a reference point for all later experiments. The purpose of the baseline was to establish a consistent starting system against which architectural and training changes could be evaluated. Using a baseline was important because it allowed us to determine whether later improvements came from genuinely better modeling choices rather than from unrelated implementation differences. All later models were therefore compared against this baseline using the same evaluation metric, CER, on the same data split.

The baseline also provided an initial sense of the difficulty of the EMG-to-text task. Because EMG signals are noisy, high-dimensional, and temporally structured, even a reasonable baseline may struggle unless it can capture both short-term local patterns and longer-range sequence information. This motivated our decision to explore more expressive sequence architectures in the next stage of the project.

2.2 Sequence Modeling Architectures

After establishing the baseline, we evaluated several alternative architectures designed to model temporal dependencies in the EMG signal more effectively. Our central hypothesis was that sequence modeling would be a major factor in performance, since the mapping from EMG activity to text depends not only on instantaneous signal values but also on how muscle activation evolves over time.

2.3 CNN Encoder + BiGRU

Our strongest architecture combined a CNN encoder with a bidirectional GRU. The CNN encoder was used to short-range patterns such as local activation bursts and channel interactions. The GRU modeled introduces memory, allowing the model to condition predictions on longer context. This is useful because keystroke-related activity is not perfectly isolated; adjacent characters can overlap in time and the signal may contain local patterns that become clearer with surrounding context. This design was motivated by the structure of the task of predicting keystrokes, the CNN can learn local spatial and short-term temporal features, while the BiGRU can integrate information over longer windows and use contextual information from both past and future frames.

We chose a BiGRU because sequence decoding can benefit from using both past and future context when forming frame-level character probabilities during training. This bidirectional context helps make the model's emissions more consistent over time and reduces noisy spikes that CTC might otherwise decode as extra characters. Overall, we expected the CNN+BiGRU combination to work well because EMG-to-text recognition needs both reliable local feature extraction and longer-range sequence context.

2.4 CNN Encoder + LSTM

We also evaluated a CNN Encoder + LSTM model. Like the BiGRU system, this model used a convolutional front end followed by a recurrent sequence model, but replaced the GRU with an LSTM. We included this comparison because LSTMs are a standard choice for sequence modeling and provide an alternative gating mechanism that may better preserve long-range information. Testing the LSTM allowed us to determine whether performance gains were due to recurrent modeling in general or whether the GRU variant was especially well matched to our task.

2.5 CNN Encoder + Transformer Encoder

In addition to recurrent models, we tested a CNN Encoder + Transformer Encoder architecture. The Transformer Encoder was included because self-attention can model long-range dependencies without relying on recurrent updates, which in principle could be beneficial for sequences with complex temporal structure. We hypothesized that the Transformer might help if important EMG cues were distributed across distant parts of the sequence.

2.6 Data Augmentation

We implemented two regularization techniques into the baseline CNN; channel dropout and additive Gaussian noise. Both methods are known to improve model generalization, preventing overfitting during training for a CNN. Our channel ablation experiment showed that performance is poor when the model has access to fewer electrode channels, which suggests that the decoder relies on distributed information across the sensor array rather than a small subset of "best" electrodes. This result motivates channel dropout as a more realistic regularizer, instead of permanently reducing the input (as in ablation), channel dropout temporarily masks a small random subset of channels during training, forcing the model to learn redundant, robust features that do not depend on any single channel while still allowing full-channel information at test time. Specifically, we added a channel dropout probability p_drop = 0.5, so that when activated, only k=4 randomly selected channels were retained from the flattened channel dimension, while all other channels are zeroed out throughout the time window. In addition, we introduced Gaussian Noise by adding noise drawn from $\mathcal{N}(0, \sigma)$ to every input value, where

$$\sigma = noise_scale \cdot std(\text{EMG}) \tag{1}$$

where noise_scale = 0.01. This was motivated by the fact that EMG recordings exhibit variability across repetitions due to factors such as electrode placement, muscle fatigue, slight articulation differences, and signal noise. If the model overfits too closely to the exact training distribution, performance may degrade on unseen examples. For v1, we choose Gaussian noise over alternatives such as uniform, impulse noise, multiplicative noise because Gaussian noise closely reflects realistic sensor noise reasonably well in EMG signals. [2]

Additionally, we also implemented another augmentation (v2), we revised the noise component used in v1 by adopting Gaussian jittering (additive, zero-mean Gaussian noise applied at each timestep). We chose Gaussian jittering because it aligns with a common sensor-noise assumption: Gaussian noise is widely used as a reasonable model for additive measurement noise in biosignals and is also employed as a baseline augmentation for sEMG in Tsinganos et al. [2]. In addition, Gaussian jittering is less destructive than impulse-like noise. Impulse noise introduces rare, large spikes that can distort the morphology of sEMG windows in an unrealistic way and may create out-of-distribution artifacts, whereas Gaussian jitter produces small, continuous perturbations over time that better preserve the underlying signal structure while encouraging robustness

2.7 Hyperparameter Tuning

We tuned the CNN Encoder + BiGRU model by sweeping learning rate and batch size, since these two hyperparameters had a large effect on training stability and final CER. In particular, we tested learning rates $\{3 \times 10^{-4}, 5 \times 10^{-4}, 1 \times 10^{-3}\}$ and batch sizes $\{16, 32\}$ while keeping the rest of the training pipeline fixed. We compared settings using validation and test CER in order to find the best optimization regime for this architecture. We included this experiment because we wanted to understand not only which model worked best, but also how much performance depended on training dynamics rather than architecture alone.

2.8 Ablation Experiments

We also ran ablation experiments to study how performance changed when useful information was removed from the system. Specifically, we tested the effects of reducing the number of EMG channels, reducing the amount of training data, and lowering the sampling rate. In each ablation, we changed only one factor at a time while keeping the rest of the setup fixed, and we measured the effect using CER. These experiments were important because they helped us analyze why certain models performed better by showing whether the system depended more on spatial information, temporal resolution, or training-data availability.

Table 1: Comparison of Different Architectures

Model	Val CER	Test CER	Δ Test CER
CNN Encoder + TDS (Baseline)	22.55	24.18	0.00
CNN Encoder+ Transformer Encoder + TDS	19.96	25.89	+1.71
CNN Encoder + BiGRU	18.22	19.21	-4.97
CNN Encoder + LSTM	18.25	19.02	-5.17

Table 2: Effect of data augmentation on CNN Encoder + BiGRU performance

Technique	Val CER	Test CER	Δ Test CER
CNN + BiGRU (no augmentation)	18.22	19.21	0.00
Augmentation v1	22.33	24.08	+4.87
Augmentation v2	17.61	18.07	-1.14

3 Results

Table 1 compares the different sequence modeling architectures. Among the models tested, the recurrent variants achieved the strongest performance, with the CNN Encoder + LSTM obtaining the lowest test CER of 19.02 and the CNN Encoder + BiGRU achieving a very similar 19.21. Both substantially outperformed the CNN Encoder + TDS baseline, improving test CER by 5.17 and 4.97 points, respectively. In contrast, the CNN Encoder + Transformer Encoder + TDS model achieved a slightly better validation CER than the baseline but a worse test CER, suggesting that it did not generalize as well in this setting. Overall, these results suggest that recurrent temporal modeling was more effective than the Transformer-based alternative in our setting.

Table 2 shows the effect of data augmentation on the CNN + BiGRU model. Augmentation v2 gave the best result, reducing test CER from 19.21 without augmentation to 18.07, while Augmentation v1 substantially worsened performance. This suggests that augmentation was not uniformly helpful, but depended strongly on the type of perturbation used. In our setting, the better-performing augmentation likely introduced variability that improved robustness without distorting the underlying EMG-to-text relationship too severely.

Table 3 summarizes the hyperparameter sweep over learning rate and batch size for the CNN Encoder + BiGRU model. Performance was highly sensitive to both settings. Very small learning rates performed poorly, while larger learning rates gave much lower CER. Batch size 16 consistently outperformed batch size 32, and the best overall result in the coarse sweep was obtained with learning rate 1×10^{-3} and batch size 16. These results suggest that the model benefited from a relatively aggressive optimization setting and smaller-batch training dynamics.

Tables 4–6 summarize the ablation experiments run on the baseline model. Across all three studies, reducing available information generally led to worse performance. Decreasing the number of channels caused a very large increase in CER, with performance degrading sharply once the full 32-channel input was reduced. Similarly, reducing the amount of training data substantially worsened results, with both the 25% and 10% settings performing far worse than the full-data condition. Lowering the sampling rate from 2000 Hz to 1000 Hz also increased CER, suggesting that temporal resolution is important for accurate decoding. Although these ablations were run on the baseline rather than the final CNN + BiGRU model, they still indicate that strong performance depends on preserving rich spatial, temporal, and training-set information.

4 Discussion

4.1 Model Architectures Compared

One of the main goals of this project was to understand which type of sequence model was best suited for EMG-to-text recognition and why. Across the architectures we compared, the CNN Encoder + BiGRU gave the strongest validation performance, outperforming the baseline as well as the CNN

Table 3: Hyperparameter tuning results for the CNN Encoder + BiGRU model

Learning Rate	Batch Size	Val CER	Test CER	Δ Test CER
3×10^{-4}	16	20.96	20.94	-3.24
3×10^{-4}	32	60.61	59.13	+34.95
5×10^{-4}	16	17.50	18.50	-5.68
5×10^{-4}	32	26.05	25.61	+1.43
1×10^{-3}	16	16.57	17.12	-7.06
1×10^{-3}	32	18.19	17.51	-6.67

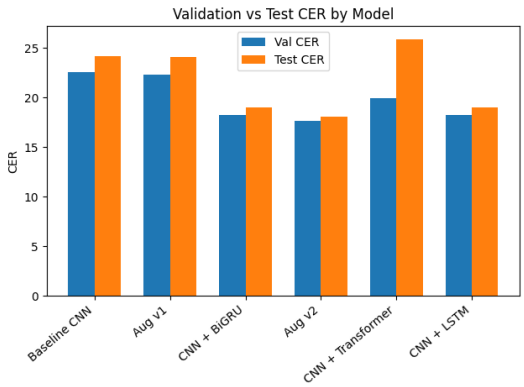

Figure 1: Validation vs Test CER by Model

Encoder + LSTM and CNN Encoder + Transformer Encoder variants. Since we based model selection off of validation CER, we chose the CNN Encoder + BiGRU as the most effective model.

A likely reason the CNN Encoder + BiGRU worked best is that it matched the structure of the EMG signal well. The CNN encoder could learn local patterns across channels and short windows of time, which is important because EMG contains fine-grained spatial and short-term temporal structure related to muscle activation. The BiGRU then helped integrate these features across the full sequence, allowing the model to use both past and future context when predicting character outputs. This is especially valuable in EMG-to-text recognition because the intended output is not determined by a single frame in isolation, but by how muscle activity evolves over time.

The weaker performance of the other architectures also provides useful insight. The LSTM-based model was still capable of sequence modeling, but it may not have provided enough additional benefit over the BiGRU to justify its heavier gating structure in this setting. In other words, the extra complexity of the LSTM may not have translated into better performance for our dataset. The Transformer Encoder was an important comparison because self-attention can in principle capture long-range dependencies very effectively. However, its lower performance suggests that this architecture may have been less well matched to the scale or characteristics of our data. One likely explanation is that the Transformer needed either more data, more tuning, or a different regularization strategy to fully benefit from attention. Taken together, these comparisons suggest that for this project, successful modeling depended not just on sequence capacity in general, but on using a temporal model that was expressive while still trainable and data-efficient.

The broader insight from the architecture comparison is that good performance came from matching the model to the signal structure. EMG is noisy and temporally complex, so the best architecture was one that could capture both short-range patterns and longer-range context without becoming unnecessarily difficult to optimize.

4.2 Data Augmentations Compared

The data augmentation experiments showed that augmentation was not uniformly helpful, but that the best augmentation strategy improved generalization and reduced CER relative to training without

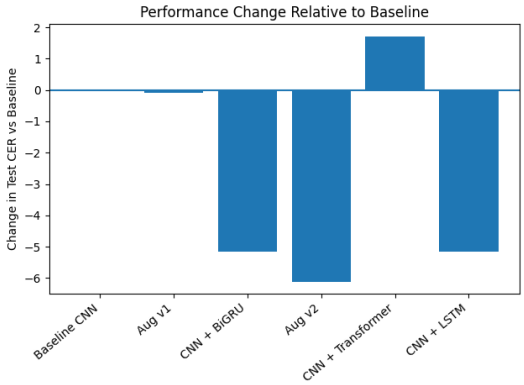

Figure 2: Performance Change Relative to Baseline

Table 4: Effect of number of channels on CER for the baseline model

Channels	**Val CER**	**Test CER**	**Δ Test CER**
32 (all)	22.55	24.18	0.00
16	77.23	76.98	+52.8
8	100.0	100.0	+75.82
4	100.0	100.0	+75.82
left only	67.12	67.54	+43.36

augmentation. This suggests that one of the main challenges in EMG-to-text recognition is not only learning the training examples themselves, but learning a representation that remains stable under natural variability in the signal.

This result makes sense given the nature of EMG data. Unlike a clean symbolic input, EMG recordings can vary for many reasons even when the intended output is the same. Small shifts in electrode position, variation in muscle activation strength, slight differences in articulation, and noise in the signal can all change the observed input. Without augmentation, the model may learn patterns that are too closely tied to the exact training distribution and therefore fail to generalize as well to held-out examples. By adding controlled perturbations during training, augmentation likely encouraged the model to focus on the more robust structure of the signal rather than overfitting to incidental details.

Another important insight is that augmentation appeared to be especially helpful for the stronger architecture. A more expressive model like the CNN Encoder + BiGRU has greater capacity to fit the training set, which means it can also benefit more from regularization. In that sense, augmentation likely worked not only by increasing apparent variability, but by acting as a form of regularization that made the learned features more invariant to small changes in the input. This is important because it suggests that performance gains did not come only from choosing a better model, but also from helping that model learn in a more robust way.

Overall, the augmentation experiments support the idea that good EMG-to-text performance depends on robustness to signal variability. The model performed better when training better reflected the kinds of variation that naturally occur in biosignal data.

4.3 Hyperparameter Tuning

The hyperparameter tuning experiments showed that performance depended strongly on both learning rate and batch size. This is an important result because it shows that architecture alone did not determine performance. Even a strong model like the CNN Encoder + BiGRU could perform poorly if it was trained under the wrong optimization settings. In our sweep, very small learning rates gave much worse CER, while larger learning rates like 1×10^{-3} performed much better. This suggests that one of the main failure modes in our project was under-training rather than instability. When the learning rate was too small, the optimizer likely made progress too slowly for the model to learn strong feature representations and good CTC alignments within the available training budget.

Table 5: Effect of training data amount on CER for the baseline model

Data amount	Val CER	Test CER	Δ Test CER
100%	22.55	24.18	0.00
25%	93.58	94.79	+70.61
10%	94.62	89.35	+65.17

Table 6: Effect of sampling rate on CER for the baseline model

Sampling rate	Val CER	Test CER	Δ Test CER
2000 Hz (baseline)	22.55	24.18	0.00
1000 Hz	63.03	50.90	+26.72
500 Hz	—	—	—

Batch size also had a clear effect. Across the settings we tested, batch size 16 consistently outperformed batch size 32. One likely reason is that the smaller batch size introduced slightly noisier gradient updates, which may have helped both optimization and generalization. For a task like EMG-to-text recognition, where the input is noisy and the model must learn alignments under the CTC objective, that additional gradient noise may have helped the model avoid poorer local solutions. It is also possible that batch size 16 simply paired better with the learning-rate range we explored, leading to more effective updates overall.

The broader insight from the hyperparameter sweep is that good performance depended not only on choosing the right model, but also on training that model with the right hyperparameters. The CNN Encoder + BiGRU worked best when paired with a relatively aggressive learning rate and a smaller batch size, suggesting that optimization quality was a major contributor to final performance. In other words, the gap between models partly came from how easily each model could be trained effectively.

4.4 Ablation Experiments

Across all three ablations, performance worsened when useful information was removed from the input or training setup. In the channel ablation, reducing the number of EMG channels caused a large increase in CER. This indicates that the model relied on information distributed across multiple electrodes rather than only a small subset of channels. That result makes sense because different channels capture different spatial aspects of muscle activation, and reducing the channel count removes part of that spatial structure.

The training-data ablation showed that using much less training data substantially hurt performance. Both reduced-data conditions were much worse than the full-data setting overall. This suggests that the model depended strongly on dataset size in order to learn robust representations.

The sampling-rate ablation showed that reducing temporal resolution hurt performance. Lowering the sampling rate from 2000 Hz to 1000 Hz substantially increased CER, suggesting that fine-grained timing information is important for accurate decoding. We could not obtain results for 500 Hz or smaller downsampling factors because those runs produced preprocessing or training errors, likely because aggressive downsampling made the input incompatible with parts of the existing pipeline. Even so, the degradation at 1000 Hz already suggests that the model depends heavily on preserving temporal detail, since short-duration changes in muscle activity likely help distinguish between similar character sequences.

One limitation is that these experiments were only run on the baseline model and not on the final CNN Encoder + BiGRU system. We chose not to repeat them on CNN + BiGRU because the baseline ablations already caused such large performance drops that they were sufficient to show these reduced-information settings were unlikely to be helpful. Even without rerunning them on the stronger model, the results still show a clear pattern: performance worsened whenever the signal or training setup became less informative. The broader insight is that good EMG-to-text recognition depends not just on architecture, but on preserving as much useful spatial, temporal, and training-distribution information as possible.

5 References

[1] Viswanath Sivakumar, Jeffrey Seely, Alan Du, Sean R Bittner, Adam Berenzweig, Anuoluwapo Bolarinwa, Alexandre Gramfort, and Michael I Mandel. emg2qwerty: A large dataset with baselines for touch typing using surface electromyography, 2024.

[2] Tsinganos, P.; Cornelis, B.; Cornelis, J.; Jansen, B.; Skodras, A. Data Augmentation of Surface Electromyography for Hand Gesture Recognition. Sensors 2020, 20, 4892. https://doi.org/10.3390/s20174892

[3] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation, 2014.

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. Advances in Neural Information Processing Systems, 2017.

[5] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. Interspeech, 2019.

[6] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks. In Proceedings of the 23rd International Conference on Machine Learning, 2006.